

Omnichannel Meta Platform — Architecture Intelligence & Execution Blueprint

Clinic AI Automation Platform • Phase 0 Deliverable

Research date: 24 August 2026

Table of Contents

Document Control

Field	Value
Deliverable	Phase 0 — Architecture Intelligence, Feasibility & Execution Plan
Scope	Evolution of an existing WhatsApp clinic AI automation system into a unified Meta omnichannel platform (WhatsApp + Instagram + Facebook Messenger)
Status	Complete pending three inputs (see Section 21H)
Research basis	Current Meta developer documentation and corroborating sources, reviewed 24 August 2026
Implementation status	No code written. No implementation performed.

Evidence labelling used throughout

Label	Meaning
VERIFIED	Official Meta documentation or direct corroboration
REC	Architectural recommendation
ASSUMPTION	Reasoned but unconfirmed
UNKNOWN	Not determinable from available information
VERIFY	Must be confirmed against live Meta docs / App Dashboard before coding

0. Read This First — Two Blocking Findings

Finding 0.1 — No existing-system artifacts were supplied

Part 1 of the brief states that information about the existing WhatsApp system would be provided. Nothing was attached. Per the instruction “If information is missing, DO NOT GUESS”, the entire “EXISTING ARCHITECTURE — CONFIRMED” section is empty. Part 1 is therefore delivered as a **structured extraction protocol** rather than a reconstruction. This is not a failure of the exercise — it is the correct output, and Phase 2 of the roadmap exists specifically to close it.

Finding 0.2 — The most consequential research result is policy, not architecture

Meta closed most out-of-window messaging paths on Messenger and Instagram during 2026. Effective 27 April 2026, API requests containing the message tags `CONFIRMED_EVENT_UPDATE`, `ACCOUNT_UPDATE` and `POST_PURCHASE_UPDATE` return error code 100, with Meta directing developers to migrate to Utility Templates or the Marketing Messages API. (**VERIFIED**)

For a clinic, this means **appointment reminders — arguably the highest-value automation in the product — do not port cleanly from WhatsApp to Instagram or Messenger**. This is covered in Part 4 and Part 5 and must be surfaced to the client before commercial commitments are made.

Part 1 — The Existing Product

1.1 Existing Architecture — CONFIRMED

Nothing.

No repository, schema, diagram, environment listing, dependency manifest, webhook configuration, or App Dashboard screenshot was provided.

The only inference safe to draw is definitional, not architectural: the system is described as connected through Meta's developer ecosystem and WhatsApp Cloud API, which means at minimum that a Meta App exists, a WhatsApp Business Account (WABA) is attached to it, at least one business phone number is registered, and some HTTPS endpoint is receiving webhook callbacks. Everything downstream of that endpoint is unknown.

1.2 Existing Architecture — REQUIRES CONFIRMATION

This is the extraction checklist for Phase 2, written so a developer with repository access can complete it in a single working session. Each item is scored for **omnichannel blast radius** — how much the plan changes depending on the answer.

A. Meta asset layer — BLAST RADIUS: CRITICAL

#	Question	Why it decides the architecture
A1	Meta App ID, app type (Business vs other), and which Business Portfolio owns it	Determines whether the same app can host Page/IG products or whether a new app is needed
A2	Is the App in Development or Live mode?	Live mode and Advanced Access are separate gates; both required for real patients
A3	Which permissions are granted, at Standard or Advanced access?	<code>whatsapp_business_messaging</code> at Standard only works for app-role holders
A4	Is Business Verification complete on the portfolio?	Verification is portfolio-level and a shared prerequisite for Page/IG Advanced Access
A5	Token type in production: user token, System User token, or Business token?	User tokens expire; the leading cause of silent production outages
A6	Is the WABA owned by the clinic's portfolio or the agency's?	Determines Direct Developer vs Tech Provider path — different App Review scope entirely
A7	Does the clinic own a Facebook Page and an Instagram Professional account, and are they linked?	Decides which Instagram integration path is available
A8	Current messaging tier and quality rating on the WhatsApp number	Affects reminder volume ceilings

B. Ingress and transport — BLAST RADIUS: HIGH

#	Question
B1	Webhook callback URL(s) — one endpoint or several?
B2	Is <code>X-Hub-Signature-256</code> HMAC verification actually implemented, or is the endpoint unauthenticated?
B3	Is webhook processing synchronous (parse-and-reply inside the request) or does it enqueue?
B4	Is there idempotency / deduplication on <code>wamid</code> ?
B5	Retry and dead-letter behaviour?
B6	Is the raw Meta payload persisted before parsing?

B3 is the single highest-value question in this entire document. If the existing system replies to WhatsApp inside the webhook HTTP request, the whole omnichannel effort begins with a queue-decoupling refactor, because Instagram and Messenger add latency and failure modes that synchronous handling will not survive.

C. Domain model — BLAST RADIUS: CRITICAL

#	Question
C1	Is there a Patient/Contact entity distinct from a phone number, or is <code>wa_id</code> the primary key?
C2	Is there a Conversation entity, or only a flat message log?
C3	Does the message table have a <code>channel</code> column, even unused?
C4	How is media stored — Meta media ID reference, or downloaded to own storage?
C5	Appointment schema — does it reference a patient or a phone number?
C6	Are conversation state and AI state persisted, or held in memory/cache?

C1 determines whether patient identity is a refactor or a rewrite.

D. AI layer — BLAST RADIUS: HIGH

#	Question
D1	Model provider; single-shot calls or agentic/tool-calling?
D2	Are tools (booking, availability) defined as callable functions, or is intent handled by prompt-only plus regex?
D3	Where does the prompt live — code, database, or config?
D4	Does the prompt contain WhatsApp-specific formatting instructions (e.g. "reply with numbered options 1-3")?
D5	How is conversation history assembled for context?
D6	Is there any guardrail or validation on AI-proposed bookings?

D4 matters more than it looks: WhatsApp interactive-button semantics differ from Instagram quick replies, and prompt-embedded channel assumptions are a common hidden coupling.

E. Business logic — BLAST RADIUS: MEDIUM

#	Question
E1	Is appointment logic in its own module/service, or inline in the WhatsApp handler?
E2	Where do doctor schedules, slots and clinic hours live?
E3	Is there an external EMR/PMS integration, or is this system the source of truth?
E4	Reminder scheduling mechanism (cron, queue, external scheduler) and which template(s) it uses?
E5	Business rules for cancellation and reschedule windows?

F. Admin surface — BLAST RADIUS: MEDIUM

#	Question
F1	Does an admin panel exist; server-rendered or SPA + API?
F2	Auth mechanism, and do roles exist?
F3	Can staff reply manually today, and does that pause the AI?
F4	Is there any audit log?

G. Operations — BLAST RADIUS: MEDIUM

#	Question
G1	Language, framework, runtime and versions
G2	Datastore(s)
G3	Hosting, deployment method, environment count
G4	Secret management
G5	Test coverage — is there any automated test?
G6	Observability: logs, metrics, error tracking
G7	Is there a staging environment with its own Meta test assets?

G5 and G7 gate Part 18 entirely. AI coding agents cannot be safely constrained without a test suite and a non-production Meta environment.

1.3 Provisional reusability classification (REC)

Pending answers, the expected split is:

- **Almost certainly channel-agnostic already:** appointment and slot logic, doctor and clinic data, knowledge-base content, business rules.
- **Almost certainly WhatsApp-specific:** webhook payload parsing, `wamid` handling, template send calls, interactive-button construction, media download via Meta media IDs, `wa_id` as identity.
- **The dangerous middle — where projects fail:** conversation state machines that encode WhatsApp message shapes; AI prompts with embedded channel formatting; the patient record keyed on phone number; anything that treats “the 24-hour window” as a universal concept when each channel’s window has different semantics and different escape hatches.

Part 2 — Meta Ecosystem Research

2.1 WhatsApp Cloud API

Asset chain (VERIFIED). Meta developer account → Meta App (Business type) → WhatsApp product added → Business Portfolio attached → WABA → business phone number. Meta's Get Started guide describes adding the WhatsApp product to a business app, attaching a business portfolio, and generating an access token from WhatsApp > API Setup, noting that production use normally relies on a system or business token rather than the user token used for the walkthrough.

Tokens (VERIFIED). System User tokens are the production-correct choice; user tokens are short-lived and unsuitable. Scopes are `whatsapp_business_messaging` (send/receive) and `whatsapp_business_management` (templates, WABA configuration).

Webhooks (VERIFIED). Subscription is on the `whatsapp_business_account` object. Verification is the GET challenge (`hub.mode`, `hub.verify_token`, `hub.challenge`); payload authenticity is the X-Hub-Signature-256 HMAC-SHA256 header computed with the app secret. Meta documents four distinct notifications for a simple exchange — sent, delivered, read, and the inbound message content itself.

Messaging model (VERIFIED). An inbound user message opens a 24-hour customer service window in which free-form messages are allowed; outside it, only approved templates. Templates are categorised (utility / marketing / authentication) and require pre-approval.

Volume and throughput (VERIFIED; figures REQUIRE VERIFICATION at build time). Cloud API default throughput is documented at roughly 80 messages per second per phone number, with upgrade possible. The messaging limit constrains business-initiated conversations to unique users in a rolling 24-hour window — commonly 250 for a new or unverified setup. Higher tiers raise the daily cap while throughput remains at approximately 80 MPS unless the account reaches the unlimited tier, where it can rise to 1,000 MPS. Quality rating drives tier movement in both directions.

Typing indicators and read receipts (VERIFIED). Now supported. Meta documents marking a user's message as read, producing blue check marks, and displaying a preparing-a-response indicator when a reply will take a few seconds. The indicator clears when you respond or after 25 seconds, whichever comes first, and marking a message read also marks earlier messages read.

App Review (VERIFIED) — a pivotal branch. Meta states that if you are using the API for yourself as a Direct Developer you do not need Advanced access or App Review, whereas an app that other businesses will use must request Advanced access for the permissions it needs. Review evidence takes the form of screen recordings demonstrating each permission in use — sending and receiving a message for `whatsapp_business_messaging`, creating a template for `whatsapp_business_management` — with an average turnaround of about 24 hours.

Policy — two hard constraints for a clinic (VERIFIED)

1. **Health data.** The WhatsApp Business Messaging Policy instructs businesses not to ask people to share sensitive identifiers, and not to use WhatsApp for telemedicine or to send or request health-related information where applicable regulations prohibit distributing such information to systems lacking heightened handling requirements. Appointment logistics are fine; clinical content is not.
2. **General-purpose AI.** Meta prohibits general-purpose AI chatbots on the WhatsApp Business Platform from 15 January 2026 — applying immediately to accounts registered on or after 15 October 2025 — while permitting AI bots built for specific structured business tasks such as customer service, order enquiries and appointment management. A scoped clinic assistant is compliant; an open-domain LLM front-end is not. This is an **architectural requirement**, not a legal footnote: the orchestrator must enforce topical scope, and you should be able to demonstrate that it does.

2.2 Instagram messaging

Two integration paths exist, and choosing between them is a Phase 1 decision (VERIFIED). Meta documents the Instagram API with Instagram Login as allowing apps to send and receive messages for Instagram Business and Creator accounts; the Instagram API with Facebook Login for Business as covering accounts linked to a Facebook Page; and Messenger API support for Instagram (referred to in Meta's developer policies as the Instagram Messaging API) as allowing Instagram professional accounts to manage both their Instagram and Facebook Page messages through one platform, requiring an Instagram Business or Creator account linked to a Facebook Page.

Dimension	Instagram API with Instagram Login	Messenger API support for Instagram (Page-linked)
Facebook Page required	No	Yes
Token	Instagram-scoped user/business token	Page access token
Messaging permission	instagram_business_manage_messages	instagram_manage_messages on the Page token
Webhook object	instagram	page / instagram topics on the Page subscription
Shares plumbing with Messenger	No	Yes

REC — use the Page-linked path. Since Facebook Messenger is a required channel anyway, the clinic must have a Page. Routing Instagram through the Page token collapses two adapters onto one auth model, one token refresh path, and one webhook signature scheme. The Instagram-Login path is architecturally attractive only if you later serve Instagram-only clinics with no Page.

VERIFY: confirm the current permission name and webhook field set in the App Dashboard at build time — Meta reorganised these docs during 2025–26 and the two paths' permission strings are easily confused.

Access levels (VERIFIED). Standard Access is the default and is intended for apps used only by people who hold roles on them. Advanced Access is required when an app serves Instagram professional accounts you do not own or manage and can be used by people without a role on the app or on a business portfolio that has claimed it; it requires both App Review and Business Verification. Meta also warns that some features may not behave correctly under Standard Access.

Messaging window and mechanics (VERIFIED). Your app can only message an Instagram user after that user has messaged the professional account, and you have 24 hours to respond. Group messaging is unsupported; an account converses with one customer per conversation; messages in the Requests folder inactive for 30 days are not returned by API calls; and for shared media only the URL appears in the webhook notification. Messages delivered via webhook or API are not treated as read in the Instagram app inbox until a reply is sent, and inbox folders are not exposed through the API.

Rich message support (VERIFIED). Quick replies, ice breakers, generic template, product template and the Handover Protocol are available. Quick replies support a maximum of 13 buttons, each truncating after 20 characters, are plain text only, post the tapped button's title into the conversation, deliver an optional payload via the messages webhook, and are not available on desktop. Ice Breakers allow a maximum of four questions, support locale-based localisation, are mobile-app only, and appear only once for first-time users; a Persistent Menu is also available. Private replies to comments do not open the 24-hour standard messaging window.

Human agent extension (VERIFIED). Meta's Instagram Platform overview documents a Human Agent feature allowing a human agent to respond using the `human_agent` tag within 7 days of a user's message, for cases where the issue cannot be resolved inside the standard window. Note the public confusion on this point — some third-party guides claim the 7-day extension is Messenger-only, while

Meta's own Instagram overview lists it. **VERIFY** that it appears as an approvable feature in your App Dashboard before designing SLAs around it.

2.3 Facebook Messenger

Asset chain (VERIFIED). Meta App → Messenger product → Facebook Page → Page access token (pages_messaging, typically alongside pages_show_list and pages_manage_metadata). Webhook object is page; the app must be subscribed to the specific Page.

Access level (VERIFIED). pages_messaging requires Advanced Access for production. Under Standard Access only app admins, developers and testers are functional — the app appears to work perfectly in development and then fails for real patients. Budget for this as a scheduled gate, not a surprise.

Messaging windows (VERIFIED). Meta's Messenger policy documents a 24-hour standard messaging window, message tags for approved out-of-window use cases, and a Human Agent tag permitting businesses to respond manually within a 7-day period; it also notes that some tags are available only on Messenger and not on the Instagram Messaging API. Meta's send-message documentation reiterates that the Human Agent tag exists specifically for escalation paths requiring manual response within 7 days.

2026 deprecations — the clinic-critical part (VERIFIED). As of 27 April 2026 the three general-purpose update tags are rejected with error code 100, and Meta's guidance is to migrate to Utility Templates or the Marketing Messages API. Third-party operators additionally report that Recurring Notifications sunset earlier in 2026 with limited regional exceptions — **VERIFY; treat as unconfirmed until read in Meta's own changelog.**

Handover Protocol (VERIFIED). Meta's Handover Protocol passes thread control between *Meta apps* (primary/secondary receiver). It is **not** your internal AI-to-human switch. You need it only if clinic staff will answer from Meta Business Suite / Page Inbox rather than from your portal. **REC:** do not adopt it in v1 — build the internal switch and treat Handover Protocol as an optional later integration.

2.4 Cross-cutting

- **Business Verification** is portfolio-level and a shared prerequisite across WhatsApp Advanced Access, Page and Instagram Advanced Access (VERIFIED). Do it once, early. Meta's processing time varies by region and can take several weeks.
- **App secret** is app-level, so one HMAC verification key serves all three webhook objects (VERIFIED by construction).
- **Webhook callback URLs** are configured per product/object in the App Dashboard — **VERIFY** the exact per-object configurability in the current dashboard.

Part 3 — The Real Connection Model

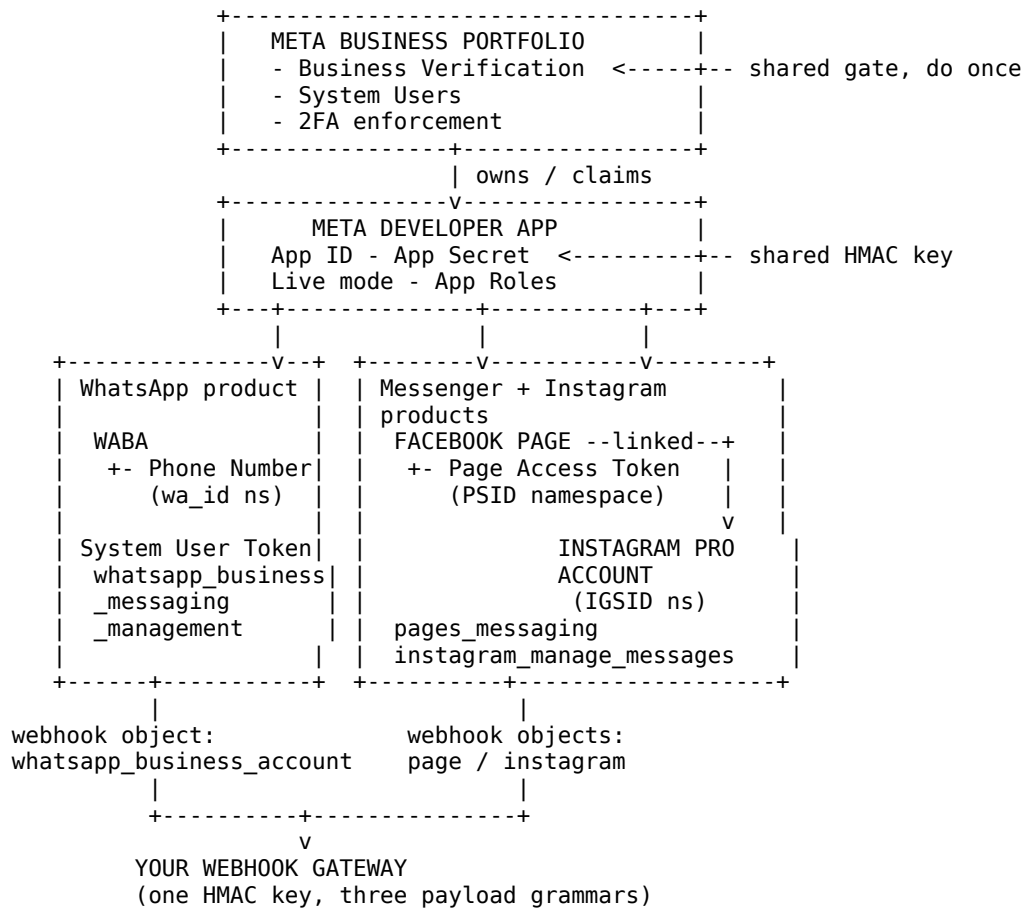
3.1 The answer

Yes — one Meta Developer App can serve all three channels (VERIFIED, with conditions).

But “one app” is a container, not a unification. The three channels share exactly four things: the App ID, the app secret, the Business Portfolio, and Business Verification status. They share **nothing else** — not the asset chain, not the token type, not the permission set, not the App Review submission, not the identity namespace, not the messaging rules.

The correct mental model: **one app, one portfolio, three independent asset spines.**

3.2 Relationship map



3.3 Where the three genuinely differ

Dimension	WhatsApp	Instagram	Messenger
Root asset	WABA + phone number	IG Professional account	Facebook Page
Token subject	System User / business token	Page token (recommended path)	Page token
Key permission	whatsapp_business_messaging	instagram_manage_messages	pages_messaging
User identifier	wa_id (phone-derived)	IGSID (scoped)	PSID (page-scoped)
Webhook object	whatsapp_business_account	instagram	page
Out-of-window escape	Approved templates	Very limited	Utility/Marketing templates + human_agent
App Review submission	Separate	Separate	Separate
Failure mode if under-	Send errors	Silent — works for testers only	Silent — works for testers only

Dimension
provisioned

WhatsApp

Instagram

Messenger

The trap. WhatsApp fails loudly, with error codes on send. Page and Instagram fail *quietly* — Standard Access produces a system that works flawlessly for the development team and does nothing for patients. Your test strategy must include at least one non-role-holding human on each channel.

Part 4 — Feasibility Verdict

PARTIALLY FEASIBLE — with the restriction concentrated in one place: proactive outbound messaging

4.1 Fully feasible (VERIFIED)

- One AI engine serving all three channels. Nothing in Meta’s model requires per-channel bots.
- One unified inbox and one admin portal. These are entirely your software; Meta imposes no constraint.
- One CRM, one appointment engine, one conversation model.
- Patients staying in native Meta apps.
- Inbound-triggered conversational automation on all three channels, inside each channel’s window.
- Human takeover with AI pause, on all three.

4.2 Restricted by Meta (VERIFIED)

R1 — Proactive outbound is not a shared capability

This is the core limitation.

Outbound need	WhatsApp	Messenger	Instagram
Appointment reminder 24h before	Approved utility template — YES	Utility Templates / Marketing Messages API; legacy tags dead since 27 Apr 2026 — CONSTRAINED	No reliable compliant mechanism — NO
Post-visit follow-up	Template — YES	As above — CONSTRAINED	NO
Human reply after 24h	Free-form until window closes, then template	human_agent tag, 7 days — YES	human_agent feature, 7 days — YES (VERIFY)
Automated reply after 24h	Template only	Not permitted under human_agent	Not permitted

Meta’s own policy is explicit that some tags exist only on Messenger and not on the Instagram Messaging API. The practical consequence: **an Instagram-originated patient cannot be reminded on Instagram.**

R2 — Cross-channel identity is not resolvable by Meta

wa_id, PSID and IGSID are separate namespaces with no Meta-provided join. Any linkage must be established by your system, by consent. See Part 10.

R3 — Instagram is capability-thinner

No message templates. Private replies to comments do not open the standard window. Requests-folder threads inactive for 30 days drop out of API results, and read semantics differ from the native app.

R4 — Health-data policy caps the product

Meta’s policy restricts telemedicine and health-information exchange where regulations impose heightened handling requirements. The platform must be positioned as *scheduling and clinic-information automation*, not clinical communication. Architecturally: no diagnoses, no results, no prescriptions in message bodies, and a redaction/blocking layer if staff attempt it.

R5 — Mitigation, and it should be a product feature, not a workaround (REC)

Build a **channel-capability-aware notification router**. When an appointment is booked from Instagram or Messenger, the AI asks — inside the open window, with explicit consent — for a

WhatsApp number or SMS/email address for reminders. The consent capture is logged; the reminder is delivered on a channel that supports it.

This turns a Meta limitation into a deliberate, auditable identity-linking moment. **It is the single most important design decision in this document.**

4.3 Not feasible (VERIFIED)

- Guaranteed reminder delivery on the patient's originating channel for Instagram and Messenger.
- Automatic identity unification across channels without patient action.
- An open-domain assistant on WhatsApp. Meta permits AI focused on specific business functions such as appointment management, not open-domain conversation.
- Reading historical conversations that pre-date your app's webhook subscription.

Part 5 — Channel Capability Matrix

Capability	WhatsApp	Instagram	Messenger	Unified approach	Meta limitation / notes
Receive text	Yes	Yes	Yes	Normalize to <code>text</code>	—
Send text	Yes, in 24h window	Yes, in 24h window	Yes, in 24h window	Window-aware send gate	All three enforce 24h (VERIFY)
Receive media	Yes	Yes	Yes	Normalize to <code>attachments[]</code>	IG shares deliver URL only
Send media	Yes	Yes	Yes	Adapter-specific upload	Format/size differ (VERIFY)
Buttons	Interactive reply buttons (VERIFY count)	Via generic template	Template buttons	Capability descriptor drives rendering	Different constructs, not interchangeable
Quick replies	List / reply-button analogue	Max 13, 20 chars, plain text, not on desktop	Yes	Emit abstract <code>choices[]</code> ; adapter degrades	Truncation rules differ
Templates	Pre-approved, categorised	None	Utility / Marketing Messages	Template registry with channel availability flags	Legacy tags return error 100 from 27 Apr 2026
Read status	Delivered + read webhooks	Differs from app semantics	VERIFY	Optional per-channel field	API-delivered IG messages are not read until you reply
Typing indicator	Yes	Sender actions	Sender actions	Best-effort, non-blocking	WhatsApp indicator clears on reply or after 25s
Conversation metadata	Minimal (<code>wa_id</code> , profile name)	IGSID; username may need separate lookup	PSID + profile fields	Own it internally	Never depend on Meta for CRM data
Webhook events	Rich	Yes	Yes	One gateway, three parsers	Shared app secret
Message history	From subscription onward only	Requests-folder threads idle 30 days drop from API	VERIFY	Your database is the history	Do not rely on Meta as archive
Human handoff	Internal	Internal + Handover Protocol	Internal + Handover Protocol	Internal state machine	Meta's protocol is app-to-app only
Automated responses	Scoped only	Yes	Yes	One orchestrator	General-purpose bots barred on WhatsApp from 15 Jan 2026
AI responses	Yes, scope-enforced	Yes	Yes	Single orchestrator, channel-blind	Scope enforcement is a compliance control
Appointment flows	Full	Inbound only	Inbound + limited outbound	Shared engine, channel-aware notification	Reminders are the fault line
Links	Yes	Yes	Yes	Universal	Preview behaviour varies
Structured interactions	Lists, buttons, flows (VERIFY)	Ice breakers (4, mobile only), persistent menu	Persistent menu, templates	Per-channel config in admin	Not portable across channels
Notifications	Templates	None	Constrained	Capability router + consent capture	Primary architectural constraint

Part 6 — Target Architecture

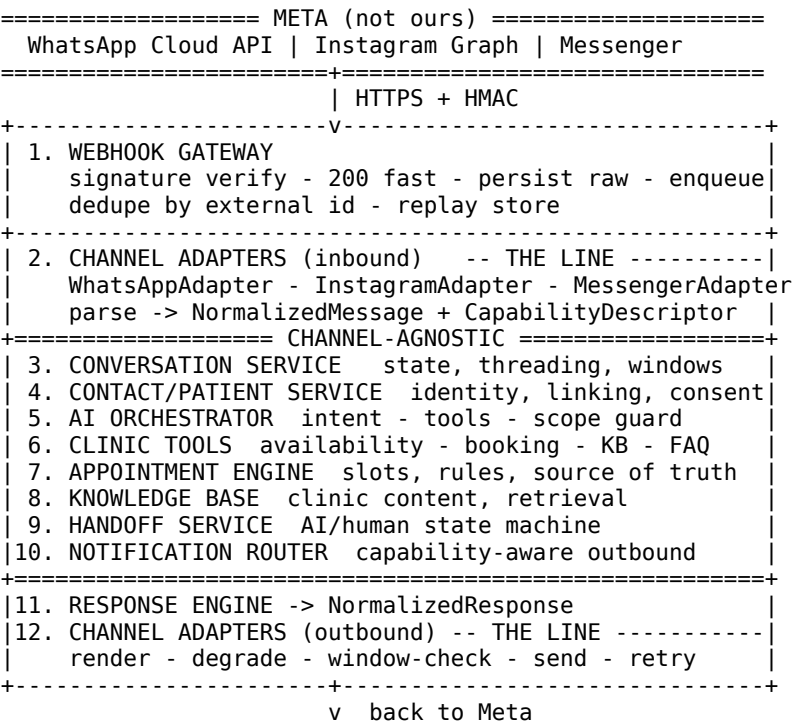
6.1 The boundary rule

Anything that knows the name of a Meta product lives above the normalization line. Nothing below the line may contain the strings “whatsapp”, “instagram” or “messenger” in a conditional.

That is the testable form of the principle. It is enforceable by lint rule and by code review, and it is the contract given to AI coding agents in Part 18.

The one legitimate exception: the business layer may consult a **capability descriptor** — an abstract statement of what the channel can do (supports_templates, window_type, max_choices) — without knowing which channel produced it. Capability-awareness is allowed; channel-awareness is not.

6.2 Layers



CROSS-CUTTING (all layers):
Auth (staff) - RBAC - Audit log - Structured logging -
Metrics/tracing - Error taxonomy - Retry + DLQ -
Token vault + rotation - Rate limiter per channel

6.3 Non-obvious component notes (REC)

The webhook gateway must be dumb. Verify, persist, enqueue, return 200. No business logic, no AI calls, no database joins. Meta retries on non-200 responses, and slow endpoints cause duplicate-delivery storms.

The raw payload store is not optional. It is your only defence when Meta changes a payload shape mid-flight, and your only replay mechanism after an outage.

The window tracker is a first-class service, not a field. Each conversation carries `window_expires_at` and `window_type`. The send path *asks permission* before every outbound message. This single component prevents the most common production failure: silent send rejections.

Token vault with proactive rotation and expiry alarms. Token expiry is the highest-frequency real-world outage in Meta integrations.

Per-channel rate limiters with distinct budgets, since throughput and conversation-initiation caps are enforced separately and exceeding either causes failures.

Part 7 — The Channel Adapter Model

7.1 Adapter contract

Every adapter implements the same interface and nothing more:

INBOUND	parse(rawEvent)	-> NormalizedMessage[]
	capabilities()	-> CapabilityDescriptor
	identify(rawEvent)	-> ChannelIdentity
	windowState(rawEvent)	-> WindowState
OUTBOUND	canSend(response, window)	-> Verdict{allow deny degrade, reason}
	render(NormalizedResponse)	-> channel payload
	send(payload)	-> SendResult
	classifyError(err)	-> ErrorClass{retryable fatal window rate}

classifyError deserves emphasis. Meta returns many error codes; treating them uniformly means retrying unretryable sends and giving up on transient ones. Each adapter owns its error taxonomy and maps to a shared internal set.

7.2 Capability descriptor

```
{
  channel_key,
  window: { type, duration_hours, extension: {mechanism, hours} | null },
  outbound_outside_window: { supported, mechanism: "template"|"utility"|"none" },
  choices: { supported, max, label_max_chars },
  rich_cards: { supported, variant },
  media: { in: [...], out: [...], max_bytes },
  typing_indicator: bool,
  read_receipts: bool,
  proactive_notification: bool    // <- false for Instagram
}
```

7.3 Graceful degradation ladder (REC)

The response engine emits intent-level constructs. Adapters degrade deterministically:

Intent construct	WhatsApp	Instagram	Messenger
choices[8]	List message (up to 10)	Quick replies, first 8, labels truncated to 20 chars	Quick replies
choices[15]	Numbered text list + free-text parsing fallback	Same	Same
rich_card	Media + text + reply buttons	Generic template	Generic template
fallback (any)	Text + link	Text + link	Text + link

Degradation must be **logged as a business event**, not silently applied — otherwise you cannot diagnose why Instagram conversion is lower than WhatsApp.

7.4 Flow

```
Meta event
-> gateway (verify, persist, enqueue, 200)
-> adapter.parse + adapter.identify + adapter.windowState
-> NormalizedMessage + CapabilityDescriptor + ChannelIdentity
-> conversation service (thread resolution, state)
-> patient service (identity resolution)
-> AI orchestrator (scoped tools)  <- knows capabilities, never the channel
-> NormalizedResponse (intent-level)
-> adapter.canSend -> render -> send
-> SendResult persisted -> delivery/read webhooks reconcile
```

Part 8 — Unified Message Model

8.1 Universal fields

Field	Type / values	Notes
id	internal UUID	Primary key
conversation_id	FK	
channel_key	enum	whatsapp / instagram / messenger
direction	inbound outbound	
sender_type	patient ai staff system	
sender_ref	staff id	When applicable
content_type	text media choice_reply location contact system_event unsupported	
text	string	Always populated
attachments[]	{ type, storage_ref, mime, bytes, caption, source }	source: downloaded remote_url
choice_selection	{ label, payload } null	
reply_to_id	internal id	
external_id	string	Meta message id — idempotency key
external_reply_to_id	string	
sent_at	timestamp	Meta timestamp
received_at	timestamp	Our timestamp
delivery_status	pending sent delivered read failed	
failure	{ class, code, message } null	
ai_generated	bool	
degraded	bool + reason	
channel_meta	opaque JSON	Write-mostly

8.2 Design rules (REC)

1. **text is always populated**, even for media (caption or a descriptor). Downstream code should never branch on content type merely to get something readable.
2. **external_id is a plain string** and must be unique-indexed per channel — it is your idempotency key.
3. **Attachments carry a source discriminator**. WhatsApp media requires an authenticated fetch by media ID; Instagram share notifications carry only a URL. Download-versus-reference is a real behavioural difference and must be explicit.
4. **channel_meta is write-mostly**. Business logic reading `channel_meta` is a boundary violation — make this a review rule.
5. **unsupported is a real content type**. Voice notes, reels shares and stickers arrive and must not crash the pipeline. Fail into a graceful “I can’t open that here” path.
6. **Never store clinical free text in a way you cannot redact**, given R4.

Part 9 — Conversation Architecture

9.1 Model

Field	Notes
id	
channel_key	
channel_account_ref	Which WABA number / Page / IG account
external_thread_key	wa_id PSID IGSID (scoped)
contact_id	FK to Contact
patient_id	FK to Patient null
status	open pending_human snoozed resolved archived
mode	AI HUMAN PAUSED SUSPENDED
assigned_staff_id	null
window_expires_at, window_type, extension_expires_at	Window tracker fields
ai_state	{ last_intent, pending_tool, slot_context, clarification_count, scope_violations }
unread_count, last_message_at, last_patient_message_at	
first_response_at, resolved_at	SLA metrics
labels[], internal_notes[]	
created_at, updated_at	

9.2 Thread identity rule (REC)

UNIQUE (channel_key, channel_account_ref, external_thread_key)

Conversations are **per-channel by construction** — this is correct and should not be fought. Unification happens one level up, at the Contact/Patient. A patient with three channel conversations has three conversation rows and one patient record.

9.3 How the unified inbox stays unified

The inbox queries **across** conversations, joined to Patient. The UI presents:

- a single chronological list with channel badges;
- a **patient rail** that, when a linked patient is selected, shows their other-channel conversations as switchable tabs;
- a “same patient, other channel” affordance that never merges timelines.

REC — do not merge message timelines across channels. It is tempting and it is wrong: the patient did not experience a merged conversation, staff replies must go back to a specific thread, and window state is per-thread. Merged timelines cause staff to reply into a closed window.

9.4 Window lifecycle

```
patient message -> window opens/resets (24h, all three channels)
|- inside window: free-form AI or human                ALLOWED
|- expired + human + within 7d: human_agent extension  IG/Msgr ONLY
|- expired + WhatsApp: approved template only          TEMPLATE
+- expired + Instagram + automated: BLOCKED -> queue or reroute
```

The conversation service exposes `canSendNow(conversation, message_kind)`. Everything else calls it. Nothing else reimplements it.

Part 10 — Patient Identity

10.1 Three-tier model

PATIENT (clinical / CRM record – the appointment subject)

```
|
+-- CONTACT (a person as known on one channel)
|   +-- ChannelIdentity: whatsapp / wa_id
|   +-- ChannelIdentity: instagram / IGSID
|   +-- ChannelIdentity: messenger / PSID
|
+-- VerifiedAttributes: phone (verified), email (verified), DOB, name
```

ChannelIdentity carries: channel_key, external_id, display_name, first_seen_at, link_state, link_method, link_confidence, linked_by, linked_at.

10.2 Why linking is hard (VERIFIED)

Meta provides no join. `wa_id` derives from a phone number; PSID is page-scoped; IGSID is Instagram-scoped. Two identities belonging to the same human look completely unrelated.

10.3 Linking rules (REC)

Automatic linking permitted only on:

- Exact match of a **verified** phone number — verified meaning confirmed through the WhatsApp channel itself, or via OTP. Confidence: high.
- An **appointment reference code** issued by the system and quoted by the patient. Confidence: high.

Requires explicit patient confirmation:

- A phone number typed into an Instagram or Messenger conversation but never verified. Ask, confirm, then link.
- Email match.

Never sufficient alone: name match, display-name similarity, DOB alone, geographic proximity, or temporal correlation (“someone booked from Instagram and a WhatsApp message came in a minute later”).

Never link on AI inference. The orchestrator may *propose* a link; only the identity service, acting on a deterministic rule or an explicit confirmation event, may commit one.

10.4 Failure asymmetry — the safety principle

Failure	Severity	Consequence
False split (one patient, two records)	Mild	Duplicate records, some duplicate reminders; fixable by staff merge
False merge (two patients, one record)	Severe	Patient A sees Patient B's appointments; a health-adjacent data breach; potentially a wrong-patient clinical action

Therefore: **bias hard toward splitting**. Require staff dual-confirmation for manual merges, make merges reversible (soft-link with full history), and audit-log every link and unlink with actor, method and confidence.

10.5 The consent moment

The R5 reminder-routing conversation is the natural and honest place to collect a verified phone number from Instagram and Messenger patients. Script it as a genuine value exchange — “so we can send your reminder” — capture consent explicitly, record the consent artifact, and let that verified

number drive the link. This is good product design *and* good compliance, and it converts Meta's biggest limitation into your identity graph.

Part 11 — AI Orchestration

11.1 One engine, channel-blind

The orchestrator receives: normalized message, conversation history, patient context, clinic context, and capability descriptor. It receives **no channel identifier**. If it ever needs one, the boundary has leaked.

11.2 Pipeline

NormalizedMessage

- > Safety & scope pre-filter
- > Context assembly (conversation, patient, clinic, capabilities)
- > Intent classification
- > Tool selection & argument extraction
- > Tool execution (validated, permissioned, idempotent)
- > Response generation (intent-level, capability-constrained)
- > Post-validation (grounding, scope, safety)
- > NormalizedResponse

11.3 Tool inventory and layer assignment

Tool	Layer	Notes
check_doctor_availability	Shared business	Read-only, safe
book_appointment	Shared business	Write — needs confirmation gate + idempotency key
cancel_appointment	Shared business	Write — requires identity confidence threshold
reschedule_appointment	Shared business	Write — same
get_doctor_info	Knowledge base	Read-only
get_clinic_timings	Knowledge base	Read-only
get_clinic_location	Knowledge base	Read-only
get_consultation_fees	Knowledge base	Read-only
list_services	Knowledge base	Read-only
search_fa	Knowledge base	Read-only, retrieval-grounded
request_human_handoff	Handoff service	State transition
capture_contact_for_reminders	Identity service	Consent-recorded

Every tool is channel-agnostic. Not one exception. The only channel-shaped decision — how to render a chosen time slot — belongs to the adapter.

11.4 Guardrails (REC) — requirements, not enhancements

1. **Scope enforcement.** The orchestrator must refuse off-domain requests. This is a Meta compliance requirement on WhatsApp, given Meta's allowance for AI restricted to specific business functions rather than open-domain conversation. Implement as an explicit classifier plus logged `scope_violations`, so the control is evidenceable.
2. **No clinical advice.** Hard-blocked category with a scripted redirect to clinic staff. Required by R4.
3. **Confirmation before every write.** The AI proposes; the patient confirms; then the tool executes.
4. **Grounded answers only.** Clinic facts come from the knowledge base, never from model memory. Unretrieved facts escalate.
5. **Idempotency on writes.** Duplicate webhook delivery must not create duplicate bookings.
6. **Clarification budget.** After N failed clarifications, hand off to a human rather than looping.
7. **Prompt-injection isolation.** Patient text is data. Never concatenated into instruction position; tool arguments schema-validated before execution.

Part 12 — Clinic Admin Portal

12.1 Modules

Unified Inbox · Patients · Appointments · Doctors · AI Settings · Knowledge Base · Automation Settings · Analytics · Staff Accounts · Audit Logs · **Channel Health**.

REC — add Channel Health to the list. Staff need to see at a glance that Instagram’s webhook is delivering and the Page token is valid. Otherwise the first sign of an outage is a complaining patient.

12.2 Unified inbox UX — three-pane layout

Left — conversation list

Each row: patient name (or channel display name if unlinked), channel badge (colour **and** icon — never colour alone, for accessibility), last message preview, relative timestamp, unread dot, mode chip (AI / HUMAN / NEEDS HUMAN), assignee avatar, and a **window countdown** when under four hours.

Filters: channel, mode, assignee, status, label, unread, has-upcoming-appointment. Search across message text, patient name and phone number.

Centre — message timeline

Single channel only. Bubbles distinguish patient / AI / staff / system, with AI messages visually marked — staff must always know what the AI said in their name. Delivery ticks shown.

A **persistent window banner** sits above the composer: green with countdown, amber under four hours, red when expired with the exact reason and the available action (“template only” / “human reply within 7 days” / “cannot message”).

Internal notes are inline, visually distinct, and never sendable by accident. Degraded renders show a small marker.

Right — patient rail

Identity block with link state and, if unlinked, a **Link to patient** action. Other-channel conversations as switchable tabs. Upcoming and past appointments with inline reschedule and cancel. Consents held. Labels. Recent AI intents.

Takeover controls

A prominent **Take over** button pauses the AI immediately and assigns the conversation. **Return to AI** requires a reason and shows what the AI will and will not do next. Staff typing in the composer triggers an auto-pause prompt — the most common real-world failure is a human and an AI replying simultaneously.

12.3 Non-negotiables (REC)

- Never let staff compose a message the channel will reject; disable send and explain instead.
- Never hide which messages were AI-generated.
- Never surface a merged cross-channel timeline (Section 9.3).
- Every appointment mutation from the inbox is audit-logged with actor.

Part 13 — Human Handoff

13.1 States

State	Meaning
AI	AI responds automatically
PENDING	Human requested, not yet claimed; AI silent except one acknowledgement; SLA timer running
HUMAN	Staff assigned; AI fully suppressed
PAUSED	AI off, no human assigned (manual pause / after hours)
SUSPENDED	Safety or policy stop; AI cannot resume without staff action

13.2 Transitions

From	To	Trigger	Notes
AI	PENDING	AI escalation, patient request, keyword, low confidence, clarification budget exhausted, sentiment threshold, clinical-content detection	Send one acknowledgement with realistic expectation
AI	SUSPENDED	Safety trigger, prompt-injection detection, repeated scope violations	Never auto-resumes
PENDING	HUMAN	Staff claims	Stop SLA timer
PENDING	AI	Timeout, where policy allows	Configurable; off by default
HUMAN	AI	Explicit staff resume	Requires explicit action + reason
HUMAN	PAUSED	Staff pause / end of shift	
any	PAUSED	Outside staffed hours + policy	
SUSPENDED	AI	Supervisor override only	Audit-logged

13.3 Rules (REC)

- **The AI never auto-resumes from HUMAN.** Silent resumption mid-conversation is the worst UX failure in this class of product.
- On resume, the AI is given a summary of what the human did — it must not repeat resolved questions.
- Escalation is channel-agnostic; only the *window mechanics* of the reply differ, and the send gate handles that.
- Meta's Handover Protocol (Section 2.3) is a separate, later concern and must not be conflated with this state machine.
- After hours, PENDING conversations should receive an honest expectation-setting message, and the reminder-routing consent moment (R5) is a good place to capture a callback channel.

Part 14 — Clinic Automation Layer

Automation	Channel-independent logic?	Delivery constraint
Booking	Fully	Inbound-triggered; all channels
Booking confirmation	Yes	Inside window; all channels
Reminders (24h / 2h)	Logic yes	Delivery is channel-bound — WhatsApp templates; Messenger utility templates; Instagram none
Cancellation	Yes	Inbound fine; proactive notice constrained as above
Rescheduling	Yes	Same
Lead qualification	Yes	Inbound; all channels
FAQ / clinic information	Yes	Inbound; all channels
Doctor availability	Yes	Inbound; all channels
Human escalation	Yes	All channels
Follow-up / post-visit	Yes	Same constraint as reminders
No-show handling	Yes	Same
Feedback request	Yes	Same

The pattern is uniform: decide channel-agnostically, deliver channel-aware.

Every automation produces a `NotificationIntent` — recipient, purpose, urgency, content variables, deadline. The **Notification Router** then resolves: preferred channel by consent → capability check → fallback ladder (WhatsApp template → SMS/email if consented → staff task queue).

If no compliant path exists, it must create a **staff task**, not fail silently. A reminder that vanishes is worse than one that becomes a phone call.

Part 15 — Security and Safety

Domain	Requirement (REC)
Meta tokens	Secrets manager, never env files in the repo; System User tokens in production; scheduled rotation; expiry alarms at T-14 / T-7 / T-1 days; separate tokens per environment
Webhook verification	Mandatory X-Hub-Signature-256 HMAC check with the app secret, constant-time compare, reject before parsing ; the GET verify-token challenge is not authentication
Webhook abuse	Rate limit the endpoint; drop oversized bodies; the endpoint is public and will be probed
Idempotency	Unique index on external message id; replay-safe processing
Staff authentication	MFA required; short sessions; no shared logins, which destroy audit value
Authorization	RBAC — Admin / Manager / Agent / Read-only; enforced server-side per request, never in the UI alone
Patient data	Encrypt at rest and in transit; field-level encryption for identifiers; documented retention and deletion; least-privilege database access
Health data	Policy-level block on clinical content per R4; content classifier warns staff; redaction utility available
Audit logging	Append-only; every appointment mutation, identity link/unlink, mode change, AI-setting change, KB edit, data export and permission change — with actor, timestamp, before/after
AI tool permissions	Explicit allowlist; write tools require confirmation; tools validate authorization independently of the AI's claims; no tool trusts an AI-supplied patient id without an identity check
Prompt injection	Patient text never in instruction position; schema validation on all tool arguments; detection heuristics; repeated attempts move the conversation to SUSPENDED; attempts logged as security events
Unauthorized appointment changes	Identity-confidence threshold before mutating an existing appointment; reference-code confirmation for cancellations; notify on the verified channel after any change
Human takeover	Staff identity recorded on every sent message; AI-generated messages permanently marked
API abuse	Per-IP and per-account rate limits on the admin API; pagination caps; export throttling and alerting
Rate limiting	Per-channel outbound budgets aligned to the separate conversation-initiation and per-second throughput caps; queue with backpressure
Logging	Structured, correlation-ID threaded; message bodies redacted or access-controlled in logs — clinic conversations in plaintext application logs is a breach waiting to happen

Part 16 — Implementation Journey

Dependency order is revised from the brief in three ways:

1. **Meta account readiness runs in parallel from day one**, because Business Verification can take weeks.
2. **Architecture extraction precedes account work in priority**, because it may change what you provision.
3. **Shared AI orchestration moves before the second and third channels**, because integrating Instagram against a WhatsApp-shaped AI means doing the work twice.

Phase 0 — Architecture intelligence

Objective	Establish the decision baseline
Dependencies	None
Inputs	Client brief, channel requirements
Outputs	This document, approved
Key components	Research, feasibility, capability matrix, risk register
Risks	Stale Meta documentation
Validation	Technical and client sign-off on the Part 4 verdict, specifically the Instagram reminder limitation
Definition of done	Parts 4, 5, 20 and 21 accepted in writing

Phase 1 — Meta account and API readiness (RUNS IN PARALLEL FROM DAY 1)

Objective	All Meta assets provisioned, verified, permissioned
Dependencies	Client asset access
Inputs	Clinic business documents, Page and Instagram account credentials
Outputs	Verified portfolio; Page linked to Instagram Professional account; Advanced Access for <code>pages_messaging</code> and the chosen Instagram messaging permission; separate staging assets; System User tokens in vault
Key components	Business Verification, App Review submissions, webhook configuration, test assets
Risks	Verification delay of weeks; App Review rejection; wrong Instagram integration path chosen
Validation	A non-role-holding external tester receives a reply on every channel
Definition of done	Advanced Access granted on all required permissions; staging assets independent of production

Start this on day one. Business verification processing varies by region and can take several weeks, and Instagram Advanced Access requires both App Review and Business Verification. It is the longest lead-time item in the project and it blocks launch, not development.

Phase 2 — Existing WhatsApp architecture extraction

Objective	Replace Section 1.2 with fact
Dependencies	Repository and infrastructure access
Inputs	Codebase, schema, deploy config, Meta dashboard
Outputs	Completed Part 1 answers; as-is diagram; coupling map; KEEP/REFACTOR/EXTRACT/REPLACE/ADD/DEPRECATE classification (Part 17); characterization test suite over current WhatsApp behaviour
Key components	Code read, schema dump, dependency graph, manual behaviour trace
Risks	Undocumented behaviour; no tests to characterize against
Validation	An engineer who did not write the system can trace a message end-to-end
Definition of done	Every Section 1.2 item answered or explicitly marked unanswerable; regression suite green on current behaviour

The characterization test suite is the deliverable that protects everything else. Without it, Phase 3 is a rewrite with extra steps, and Part 18's constraints on AI agents are unenforceable.

Phase 3 — Channel-agnostic foundation

Objective	Draw the boundary line without changing behaviour
Dependencies	Phase 2
Inputs	As-is architecture, target architecture
Outputs	Webhook gateway (verify / persist / enqueue / 200); queue; WhatsApp adapter extracted; business logic separated; WhatsApp behaviour byte-identical
Key components	Gateway, queue, adapter interface, capability descriptor
Risks	Regression on the live channel; scope creep into unrelated refactoring
Validation	Characterization suite green; production WhatsApp metrics unchanged for 7 days
Definition of done	Zero behaviour change, zero patient-visible difference, adapter interface defined and implemented once

Phase 4 — Unified message and conversation model

Objective	Migrate to the normalized model
Dependencies	Phase 3
Outputs	Message / conversation / contact / patient schema; migration of existing WhatsApp data; window tracker service; identity tables
Risks	Data loss in migration; identity model wrong and expensive to change later
Validation	Migration dry-run on a production clone; reversible; row counts and spot-checks reconcile
Definition of done	All WhatsApp traffic flows through the unified model; rollback tested

Phase 5 — Unified inbox (WhatsApp only)

Objective	Prove the inbox on one channel
Dependencies	Phase 4
Outputs	Three-pane inbox, staff auth, RBAC, takeover controls, window banner, audit log
Risks	Building for three channels before one works
Validation	Clinic staff run a full day of real WhatsApp traffic from the new inbox
Definition of done	Staff prefer it to whatever they used before

Phase 6 — Shared AI orchestration (MOVED EARLIER)

Objective	Channel-blind AI with the full tool set and guardrails
Dependencies	Phases 4, 5
Outputs	Orchestrator; tool layer; scope enforcement; injection isolation; confirmation gates; idempotent writes
Risks	Channel assumptions surviving in prompts
Validation	An automated test replays identical normalized inputs with three different capability descriptors and asserts equivalent decisions with differing renders
Definition of done	The orchestrator contains no channel identifier and passes the boundary lint rule

Doing this before Instagram is the single biggest schedule saving in the plan. Integrating a second channel against a WhatsApp-shaped AI means integrating it twice.

Phase 7 — Instagram integration

Objective	Second channel through the same spine
Dependencies	Phase 1 (Advanced Access), 3, 4, 6
Outputs	Instagram adapter; capability descriptor; degradation rules; inbox badge; Instagram conversations live
Risks	Wrong integration path; Standard-Access illusion; quick-reply truncation; unsupported media types
Validation	External non-tester patient completes a booking end-to-end on Instagram
Definition of done	The adapter is the only Instagram-aware code in the system; no

business-layer change was required to add it

This phase is the architecture's exam. If adding Instagram required edits below the line, the boundary is wrong — stop and fix it before Phase 8, because Messenger will find the same crack.

Phase 8 — Facebook Messenger integration

Objective	Third channel
Dependencies	Phase 7
Outputs	Messenger adapter; utility-template path for out-of-window; inbox badge
Risks	Deprecated-tag usage
Validation	External patient end-to-end; explicit test asserting the retired tags are never emitted
Definition of done	As Phase 7. Expect this to be markedly faster — if it is not, the adapter abstraction is leaking

Phase 9 — Notification router and clinic automation

Objective	Capability-aware outbound
Dependencies	Phases 6, 7, 8
Outputs	NotificationIntent model; router with fallback ladder; consent capture flow (R5); staff-task fallback; reminder / follow-up / no-show automations
Risks	Silent reminder failure — the highest-impact operational risk in the product
Validation	Deliberate test: schedule reminders for patients on each channel; assert WhatsApp delivers, Messenger delivers via utility template, and Instagram creates a consent prompt or a staff task and never silently drops
Definition of done	Zero-silent-failure guarantee demonstrated with a forced-failure test

Phase 10 — Human handoff hardening

Objective	Full state machine across channels
Dependencies	Phases 5, 6
Outputs	Five-state machine; SLA timers; assignment; auto-pause on staff typing; resume-with-summary; after-hours policy
Validation	Simulated concurrent AI/human reply attempt is prevented
Definition of done	No path exists by which the AI and a human reply to the same message

Phase 11 — Patient identity resolution

Objective	Safe cross-channel linking
Dependencies	Phases 4, 9
Outputs	Link-state model; automatic rules; confirmation flows; staff merge with dual confirmation; reversible soft-links; full audit
Risks	False merge
Validation	Adversarial test set including same-name-different-person and shared-family-phone cases
Definition of done	No automatic link on name similarity anywhere in the code; every merge reversible

Phase 12 — Security hardening

Objective	Close the Part 15 list
Dependencies	All prior
Outputs	Signature verification audit, token rotation, RBAC audit, log redaction, injection test suite, rate limiting, penetration-test remediation
Validation	External review; injection suite green
Definition of done	Every Part 15 row has an owner, an implementation and a test

Phase 13 — Testing

Objective

Dependencies

Outputs

Definition of done

Regression confidence

Phases 3–12

Adapter unit tests; contract tests per channel; capability-matrix parity suite; window-logic suite; identity adversarial suite; end-to-end per channel; load test to channel rate limits; chaos tests (webhook storm, token expiry, Meta 5xx)

CI green; every capability-matrix cell has a test or an explicit “not supported” assertion

Phase 14 — Meta production readiness

Objective

Dependencies

Outputs

Risks

Definition of done

Compliant launch posture

Phases 1, 13

App in Live mode; all Advanced Access confirmed; templates approved; privacy policy and data-deletion callback published; scope-compliance evidence pack for the AI policy; opt-in records; policy review

Launch-time discovery of a missing permission

External non-tester users function on all three channels in production

Phase 15 — Deployment and monitoring

Objective

Outputs

Definition of done

Operate it

Blue/green or canary deploy; per-channel dashboards (webhook lag, send success, window expiries, degradation rate, AI escalation rate, token TTL); alerting on token expiry, webhook silence, send-failure spikes, quality-rating drops; runbooks; on-call

An induced token expiry pages someone within five minutes

Part 17 — Implementation Strategy: Evolve, Don't Rebuild

17.1 The governing rule

WhatsApp must work, unchanged and uninterrupted, at every single commit.

Not “at every release” — at every commit. This is what makes strangling safe and what makes AI-agent implementation survivable.

17.2 Classification framework

Since the existing system is unknown, this is the **decision rule** to apply during Phase 2, not a pre-filled verdict.

Class	Definition	Expected contents
KEEP	Already channel-agnostic and working	Appointment and slot logic, clinic and doctor data, KB content, business rules, working infrastructure
REFACTOR	Same behaviour, better shape	Handlers mixing parsing with business logic; prompts with channel formatting; phone-number-as-identity
EXTRACT	Move, don't rewrite	WhatsApp API calls into the adapter; parsing into the adapter; business logic out of handlers into services; AI logic into the orchestrator
REPLACE	Cannot support omnichannel	Synchronous webhook handling — gateway + queue; flat message log — unified model; phone-as-primary-key — patient/contact/identity
ADD	Genuinely new	Instagram and Messenger adapters, capability descriptors, unified inbox, notification router, identity resolution, handoff state machine, audit log, monitoring
DEPRECATE	Remove after the new path is proven	Legacy handler entry points, direct DB access from WhatsApp code, deprecated Meta tag usage, ad-hoc token handling

Bias strongly toward KEEP. Working clinic logic embodies undocumented real-world rules; rewriting it loses knowledge you cannot recover from the code.

EXTRACT is the largest and highest-value category. Extraction preserves proven behaviour while creating the boundary.

Refactors must be behaviour-preserving and covered by characterization tests. Each replacement needs a migration plan and a rollback. Deprecate with a dated removal plan; do not delete on the same day you replace.

17.3 Strangler sequence

Stage	Action
1	New gateway in front of the existing handler. Behaviour identical.
2	Extract the WhatsApp adapter. Handler now consumes normalized messages.
3	Extract business services. Handler becomes thin orchestration.
4	Introduce the unified model behind a feature flag; dual-write; verify.
5	Cut reads to the new model. Old path idle but present.
6	Introduce the orchestrator behind a flag; shadow-mode compare; cut over.
7	Add Instagram — the first channel added with zero business-layer edits.
8	Add Messenger.

Stage	Action
9	Remove dead legacy paths.

Every stage: feature-flagged, independently reversible, validated in production before the next begins.

17.4 Two anti-patterns to name explicitly

- **The parallel rebuild.** Building “v2” alongside and switching later. It always takes longer than estimated, the two diverge, and the switch is a cliff. Reject it.
- **The big-bang model migration.** Changing the message schema in one deploy. Use dual-write and staged cutover instead.

Part 18 — AI Coding Agent Execution Methodology

18.1 Why this section is load-bearing

AI coding agents are excellent at local implementation and dangerous at architectural scope. The failure mode is not bad code — it is *helpful* code: an agent that notices the WhatsApp handler is messy and improves it while implementing something else. On a live clinic system, that is an outage.

The methodology below is designed so that **an agent’s blast radius is smaller than a single task**.

18.2 The hierarchy

```
MASTER ARCHITECTURE (this document, frozen, versioned)
+- PHASE (Part 16; human-approved entry and exit)
  +- SUB-PHASE (coherent slice, roughly one week)
    +- TASK (single-session, single-concern, testable)
      +- IMPLEMENTATION
        +- TEST
          +- VERIFICATION (automated gate)
            +- APPROVAL (human gate)
              +- NEXT TASK
```

18.3 The task contract

Every task handed to an agent must specify, in writing:

- 1. **Objective** — one sentence, one concern.
- 2. **In-scope files** — explicit allowlist.
- 3. **Out-of-scope** — explicit, including “everything not listed above”.
- 4. **Interfaces** — exact signatures the task must implement or consume; given, not invented.
- 5. **Meta API surface** — the specific documented endpoints and fields to use, with links. *The agent may not introduce any Meta API element not named here.*
- 6. **Tests required** — named cases that must pass.
- 7. **Must not change** — behaviours that must remain identical; always includes all existing WhatsApp behaviour.
- 8. **Definition of done** — objective and checkable.

A task without all eight fields is not ready to hand to an agent.

18.4 Constraint rules

Failure mode	Constraint
Random refactoring	File allowlist; diffs touching out-of-scope files are auto-rejected; “improvements” outside scope go to a backlog file, never into the diff
Destroying WhatsApp	Characterization suite runs on every commit and is a hard gate; the WhatsApp adapter and handler are protected paths requiring explicit human authorization to modify
Inventing Meta APIs	Agents may only use Meta surface named in the task contract; all Meta calls go through adapters, never inline ; a CI check greps for <code>graph.facebook.com</code> outside adapter directories and fails the build
Changing architecture	The boundary lint rule (Section 6.1) runs in CI; new packages, new dependencies and new cross-layer imports require human approval
Exposing secrets	Secret-scanning pre-commit hook; agents never receive real tokens; local and staging fixtures only; <code>.env</code> files are never in scope
Mixing tasks	One task, one concern, one PR; multi-concern diffs are rejected without review
Skipping tests	Test job is a required status check; coverage must not decrease; a task whose tests were not written does not enter review
Proceeding past failure	Gates are blocking, not advisory; on failure the agent reports and stops — it does not attempt a workaround, disable a test, or add a skip

18.5 The four gates

Gate	Type	Checks
1	Automated	Build, lint, boundary rule, no-inline-Meta-calls, secret scan, unit and contract tests, characterization suite (WhatsApp unchanged)
2	Architect review	Diff scope respected? Boundary honoured? Meta surface as specified? Adapter-only?
3	Functional	Staging test against Meta test assets on the affected channel(s)
4	Approval	Named human accepts; task closed; next task released

No agent begins task N+1 before task N clears Gate 4. This is deliberately slow at the boundary and fast within a task.

18.6 Context discipline

Give each agent: this document’s relevant sections, the task contract, the interfaces it consumes, and the tests it must pass.

Do **not** give it the whole repository and a goal. Broad context plus broad goals is precisely what produces broad, unreviewable diffs.

Part 19 — Risk Register

#	Risk	Prob.	Impact	Mitigation	Detection	Fallback
R01	Instagram cannot deliver reminders	Certain	High	Design R5 consent routing from Phase 0; set client expectations before contract	Already confirmed	Consent-captured WhatsApp/SMS; staff call task
R02	Business Verification delays launch	Med	High	Start day 1, parallel to all development	Portfolio status	Development continues on staging; launch date carries verification as an explicit dependency
R03	App Review rejection (pages_messaging / Instagram)	Med	High	Prepare screencasts exactly per permission; verify test-user access; separate submissions	Review dashboard	Resubmit with corrections; sequence a WhatsApp-first launch
R04	Standard-Access illusion — works for team, fails for patients	High	High	Mandatory external non-role-holder test on every channel before any “done”	Phase 7/8 validation	Escalate to Advanced Access; do not launch the channel
R05	Meta API or policy change mid-build	High	Med	Adapter isolation; changelog review at each phase gate; no inline Meta calls	Changelog subscription; error-rate alerts	Adapter-local fix, no business-layer change
R06	Token expiry in production	High	High	System User tokens; vault; rotation; T-14/7/1 alarms	Token TTL metric; auth-error spike	Documented rotation runbook; on-call page
R07	Webhook delivery failure or silence	Med	High	Fast-200 gateway; raw persistence; replay tooling	Time-since-last-webhook alarm per channel	Replay from raw store; Meta redelivery
R08	Duplicate processing creates duplicate bookings	Med	High	Unique index on external id; idempotent writes	Duplicate-appointment detector	Dedup job; staff merge
R09	Rate limits hit	Low–Med	Med	Per-channel budgets; queue with backpressure; respect conversation-initiation and throughput caps	Rate-limit error metric	Throttle and retry with backoff
R10	AI hallucinates clinic facts	Med	High	KB grounding only; no model-memory facts; post-validation; escalate when unretrieved	Answer-grounding audit; staff reports	Suspend AI; scripted fallback; supervisor review
R11	False patient merge	Low	Severe	No name-based linking; verified-attribute rule; dual confirmation; reversible	Merge audit review; patient complaint	Immediate unlink; incident process; notify affected
R12	Wrong appointment	Low	High	Identity-confidence	Mutation audit log	Restore from audit; staff

#	Risk	Prob.	Impact	Mitigation	Detection	Fallback
	mutated			threshold; reference-code confirmation; confirm-before- write		outreach
R13	Reminder silently not sent	Med	High	Router must never fail silently; staff- task fallback; forced-failure test	Delivery reconciliation job	Staff call list
R14	Prompt injection via patient message	Med	High	Data/instruction isolation; schema-validated tool arguments; SUSPENDED state	Injection detector; scope- violation counter	Suspend conversation; human review
R15	Clinical content sent over Meta channels	Med	High	Policy block; content classifier; staff warning; redaction tooling per R4	Classifier alerts	Redact; staff training; policy review
R16	WhatsApp AI- policy non- compliance	Low-Med	Severe (account loss)	Enforce narrow scope in the orchestrator; log scope violations as evidence	Scope-violation rate	Tighten scope; prepare compliance evidence pack
R17	Capability drift between channels confuses staff	Med	Med	Window banner; disabled sends with explanations; channel badges	Staff feedback; rejected-send rate	UX iteration
R18	Existing WhatsApp regression during refactor	Med	Severe	Characterization suite as blocking gate; feature flags; strangler stages	Production WhatsApp metrics; CI	Immediate flag rollback
R19	AI agent scope violation damages system	Med	High	Part 18 constraints; file allowlists; four gates	Diff review; CI	Revert commit; tighten task contract
R20	Quality rating drop causes tier downgrade	Low	Med	Consent hygiene; relevance; opt- out honoured	Quality rating monitor	Reduce volume; review content

Top three to brief the client on this week: R01, R02, R04.

Part 20 — Open Questions

A. For the developer / team

#	Question
A1	Are there any automated tests today? (<i>Gates Phase 2 and Part 18 entirely.</i>)
A2	Is webhook handling synchronous or queued?
A3	Is the patient record keyed on phone number?
A4	Is AI tool-calling implemented, or intent-by-prompt / regex?
A5	Is appointment logic separable from the WhatsApp handler today?
A6	Is there a staging environment, and does it have its own Meta test assets?
A7	What are the language, framework and datastore, and are they suitable for a queue-based architecture?
A8	Who owns the Meta App and the WABA — the clinic or the agency?

B. For the clinic client

#	Question
B1	Given that Instagram cannot reliably deliver appointment reminders, is the acceptable behaviour (a) ask for a WhatsApp number, (b) staff phone call, or (c) accept no reminder for Instagram-originated bookings? (<i>Blocks Phase 9 and affects Phase 0 sign-off.</i>)
B2	Does the clinic already have a Facebook Page and an Instagram Professional account, and are they linked?
B3	Expected message volume per channel — for tier planning
B4	Staffed hours, and expected response SLA after AI escalation
B5	Which topics must always go to a human?
B6	Is this platform the source of truth for appointments, or does an EMR/PMS hold that?
B7	What patient data may be discussed over Meta channels? (<i>Feeds the R4 policy block.</i>)
B8	Which jurisdiction's health-data regulation applies?
B9	Will staff work only in your portal, or also in Meta Business Suite? (<i>Decides whether Handover Protocol is needed.</i>)
B10	How is patient opt-in currently obtained and recorded?
B11	Is this one clinic, or the first of many? (<i>Direct Developer vs Tech Provider — changes App Review scope substantially.</i>)

C. Requiring Meta verification (VERIFY before coding)

#	Question
C1	Confirm the current Instagram messaging permission string and webhook fields for the chosen integration path
C2	Confirm the Human Agent 7-day feature is available and approvable for Instagram in your App Dashboard (documentation and third-party reports conflict)
C3	Confirm the exact current mechanism and regional availability for out-of-window Messenger utility messaging after the April 2026 tag deprecation
C4	Confirm whether per-object webhook callback URLs are independently configurable in the current dashboard
C5	Confirm current WhatsApp interactive-message limits (reply-button count, list-row count)
C6	Confirm current messaging tiers and throughput values against Meta's own documentation, not third-party summaries
C7	Confirm App Review evidence requirements for each permission at submission time
C8	Confirm data-deletion callback requirements for the Live-mode app

D. Requiring inspection of the existing application

All of Section 1.2, prioritized: **B3** (sync vs async), **C1** (identity key), **C2 / C3** (conversation model), **D2 / D4** (AI tooling and prompt coupling), **G5** (tests), **A3 / A4** (permissions and verification state).

Part 21 — Final Execution Blueprint

A. Executive verdict

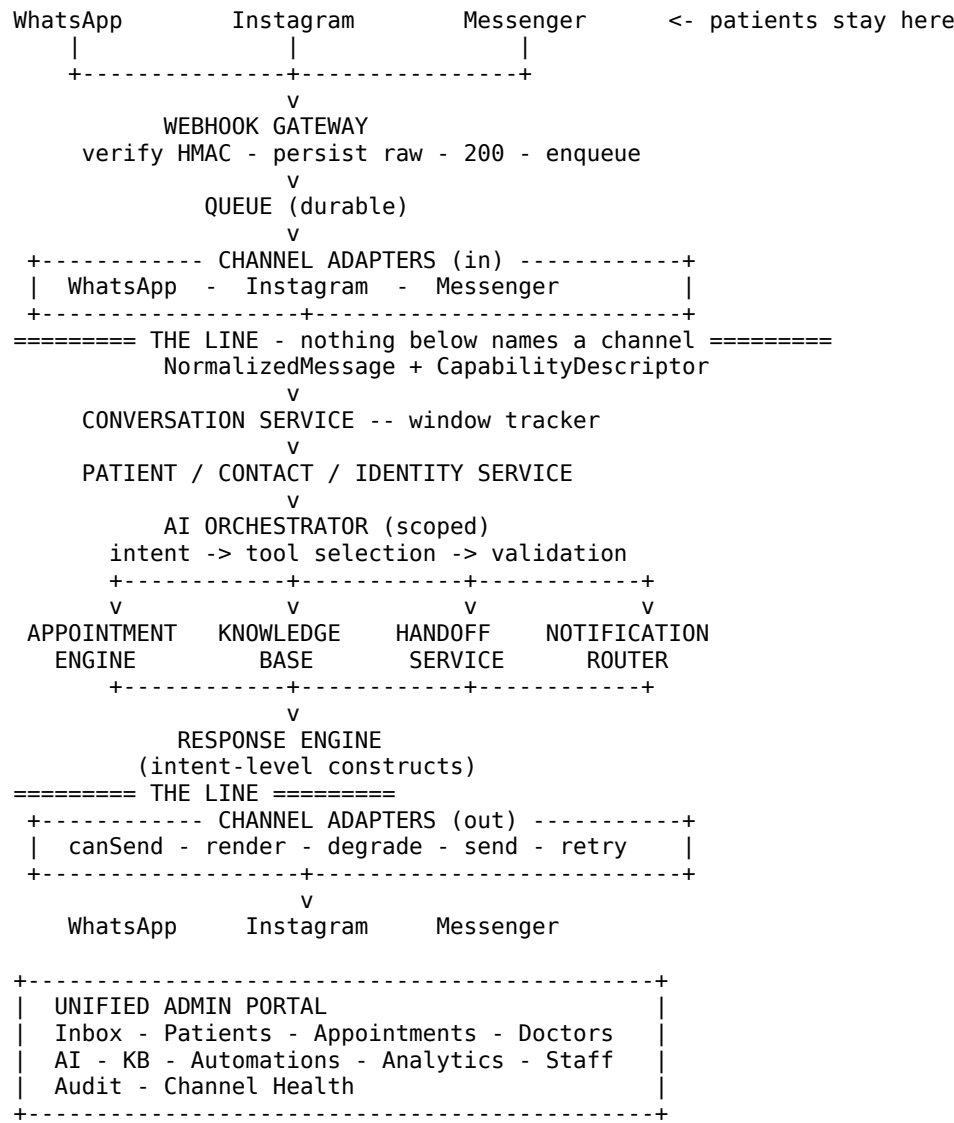
PARTIALLY FEASIBLE — build it, with one honest carve-out.

One AI engine, one conversation model, one CRM, one appointment engine, one unified inbox and one admin portal across WhatsApp, Instagram and Messenger, with patients staying in their native apps, is **fully achievable**. Nothing in Meta’s architecture prevents it, and one Meta App can host all three channels.

The carve-out is **proactive outbound**. WhatsApp templates make reminders reliable. Messenger’s path narrowed sharply in April 2026. Instagram has no equivalent mechanism. Any commitment to “the same experience on all three channels” should be corrected now, before contract, to “*the same conversation on all three channels; reminders route to a channel that supports them, with the patient’s consent.*”

Handled that way, the limitation becomes a designed feature rather than a discovered defect.

B. Final target architecture



Cross-cutting: Auth - RBAC - Audit - Logging -
Metrics - Errors - Retry/DLQ - Token vault - Rate limits

C. Meta integration map

One Business Portfolio (verified once) → one Meta App (one App ID, one app secret) → **three independent asset spines:**

Spine	Asset chain	Token	Permission	Webhook object	Identity
WhatsApp	App → WABA → phone number	System User token	whatsapp_business_messaging, whatsapp_business_management	whatsapp_business_account	wa_id
Messenger	App → Facebook Page	Page token	pages_messaging	page	PSID
Instagram	App → Page → linked IG Professional account	Same Page token	instagram_manage_messages	instagram	IGSID

Three separate App Review submissions. Three separate identity namespaces with **no Meta-provided join**.

D. System boundaries

Meta owns	You own
Message transport and delivery	All business logic
Messaging windows and policy	Conversation state and history
User identifiers within each channel	Patient identity and cross-channel linking
Message-type capabilities	Capability abstraction and degradation
Rate limits and quality ratings	Queueing, backpressure, retry
Template approval	Template content and routing
The patient's client app	The staff's entire experience

Rule: Meta is a transport, not a system of record. Your database is the history, the CRM and the truth.

E. Implementation roadmap (dependency-ordered)

Day 1 --> PHASE 1 Meta account readiness =====> (parallel, weeks)

```
PHASE 0 Architecture intelligence [COMPLETE - this document]
  +-> PHASE 2 Extract existing architecture + characterization tests
    +-> PHASE 3 Channel-agnostic foundation (WhatsApp unchanged)
      +-> PHASE 4 Unified message & conversation model
        +-> PHASE 5 Unified inbox (WhatsApp only)
          +-> PHASE 6 Shared AI orchestration <- moved earlier
            +-> PHASE 7 Instagram <- the architecture's exam
              +-> PHASE 8 Facebook Messenger
                +-> PHASE 9 Notification router + automation
                  +-> PHASE 10 Handoff hardening
                    +-> PHASE 11 Identity resolution
                      +-> PHASE 12 Security hardening
                        +-> PHASE 13 Testing
                          +-> PHASE 14 Meta production readiness
                            +-> PHASE 15 Deploy & monitor
```

F. First implementation step

Phase 2, task 1: read the existing system and write the characterization test suite — while Phase 1 (Business Verification) runs in parallel from the same day.

Not the Instagram adapter. Not the inbox. Not a line of new feature code.

This is the safest first step because it is the only one that cannot break anything, it converts Section 1.2 from questions into facts, and it produces the regression net that every subsequent phase — and every AI coding agent — depends on for safety. Starting Business Verification the same day costs nothing and removes weeks from the critical path.

G. What must not be built yet

1. **Instagram or Messenger adapters** — before the boundary exists, they will be built against WhatsApp-shaped internals and rewritten.
2. **The unified inbox for three channels** — build it for one, prove it, then extend.
3. **Cross-channel identity linking** — before the identity model and consent flow are designed, this is the R11 severe risk waiting to happen.
4. **Any reminder automation for Instagram or Messenger** — before the client answers B1.
5. **Multi-tenant / multi-clinic architecture** — before B11 is answered; it changes the entire Meta permission model.
6. **Meta Handover Protocol** — before B9 is answered.
7. **Analytics dashboards** — there is no data model to analyse yet.
8. **Any AI prompt rewriting** — before Phase 2 reveals what channel coupling already exists in the prompts.
9. **Voice, calling, or catalog features** — out of scope; do not let capability research become scope creep.

H. What must be verified before coding

Meta assumptions — all of Part 20 C. Chiefly: the Instagram integration path and its exact permission string (C1); the Instagram Human Agent 7-day feature (C2), where documentation and third-party sources genuinely conflict; and the current out-of-window Messenger mechanism after the April 2026 tag deprecation (C3). These three determine the capability matrix, and the capability matrix determines the adapter design.

Existing-system assumptions — Part 20 D. Chiefly: synchronous versus queued webhook handling, and whether the patient record is keyed on phone number. These two determine whether Phase 3 takes a week or a month.

Client assumptions — B1 above all. The Instagram reminder decision shapes Phase 9, shapes the identity model, and shapes what you can honestly promise. Get it in writing before Phase 0 is signed off.

Document Status

Phase 0 complete pending three inputs:

1. The existing-system artifacts (Section 1.2)
2. The client's answer to Question B1
3. Meta verification items C1–C3

This document is the frozen master architecture referenced by Part 18. Changes to it require explicit versioning and re-approval, and no implementation prompt should be generated against an unfrozen version.